Malware Behavior Analysis in a Sandbox

An AgentTesla Case Study

A self-relaunching .NET information stealer and its attempts to reach smtp.yandex.com on port 587

Prepared by: Fabx-crypt

Date: 16 October 2026

Tools: VirtualBox, Process Monitor, Regshot, tcpdump and tshark, INetSim, dnsmasq

Sample SHA256: `e76c5c79066d31f5e84b172d61ba575926a18505efa47a9d9a0c7c1c0ced4768`

Summary

This report describes what happened when a Windows executable from MalwareBazaar, labelled as AgentTesla, was run inside an isolated virtual lab. A Windows 10 machine played the victim, and a Kali Linux machine played the internet. Process Monitor, Regshot, packet captures and the logs of a simulated mail server recorded what the program did.

The sample is a .NET program that carries invented vendor details ("SemaphoreLine Systems"). When it is launched, it starts a second copy of itself and the original process exits, leaving the copy running. In the first run's network capture, the machine looked up smtp.yandex.com about every 20 minutes and then tried to connect on port 587, the port used to submit email. In the third run, Process Monitor tied one of those connections directly to the sample, and the simulated mail server's log shows the client asking for an encrypted session, completing it, and hanging up about two seconds later. The content of that session was encrypted and was not captured, so no stolen data was seen.

On this evidence the sample is classified as an information stealer from the AgentTesla family. The most likely operator is a financially motivated criminal using commodity tools, although the data does not allow attribution to any particular group. The report ends with an indicator list, detection and defence recommendations, and an honest account of what the lab could not show.

1. Introduction and objectives

The project looks at what a piece of Windows malware does once it is running. The brief asked for a safe sample to be executed in an isolated sandbox, for its file system, registry, network and process activity to be observed, and for the results to be turned into a classification, a list of indicators of compromise (IOCs), and a threat brief with a hypothesis about who might be behind it.

Rather than an online sandbox such as Any.run or Cuckoo, the analysis was done in a local lab built from virtual machines. That gave full control over the network, which turned out to matter, because much of what the sample did only became visible once the lab could pretend to be the internet. Wireshark was installed on the analysis machine, but the packet captures were taken on the Kali machine and read with tshark, Wireshark's command-line companion.

The objectives were to:

- build an isolated lab in which live malware could run without reaching the real network;
- obtain a real sample, confirm it is the file it claims to be, and run it;
- record process, file, registry and network behavior, and tie each observation to the sample itself;
- classify the malware, produce an IOC list, and form a cautious hypothesis about the actor behind it;
- report honestly on what the evidence does and does not show.

2. Lab environment

2.1 Machines and tools

The lab ran in Oracle VirtualBox on a Windows host. Two virtual machines did the work. The Windows 10 machine was the victim and held the sample and the monitoring tools. The Kali Linux machine acted as the analyst workstation, the file server used to move the sample across, and the fake internet. A third virtual machine, Ubuntu Desktop, was not used.

Tool	Where	Used for
Process Monitor (Sysinternals)	Win10	Process tree, plus file, registry and network events for each process
Regshot 1.9.0	Win10	Before and after comparison of the registry and user folders
Wireshark with Npcap	Win10	Installed on the victim; captures were taken on Kali instead
tcpdump and tshark	Kali	Recording and reading packet captures
INetSim	Kali	Simulated HTTP, HTTPS, SMTP, FTP and POP3 services, with logs
dnsmasq	Kali	Answering every DNS lookup with Kali's own address
VirtualBox snapshots	Host	Returning Win10 to a clean state between runs

2.2 Isolation

Before any malware touched the machine, the Win10 VM was cut off from everything except Kali:

- **Network.** Only the host-only adapter stayed enabled. The NAT adapter, which had been used to download the tools, was switched off.
- **Host channels.** The shared clipboard and drag-and-drop were disabled, shared folders were removed, and the USB controller was turned off (the pointing device was changed to PS/2 so that this was possible).
- **Antivirus.** Windows Defender's real-time protection, cloud protection, sample submission and tamper protection were switched off inside the VM, and the analysis folder was excluded. Without this, Defender would have quarantined the sample on arrival. This was done only inside the isolated VM.
- **Snapshots.** The VM was saved at five points: Post-Update Clean, Tools-Ready, Isolated-Ready, Sample-Staged and Ready-To-Detonate. The last one was restored before each run.

2.3 The simulated internet

The host-only network does not hand out addresses, so the two machines were given fixed ones: 192.168.56.10 for Kali and 192.168.56.20 for Win10. The sample was moved onto Win10 over this link. A small web server on Kali offered the zip file, Win10 downloaded it with curl, and a firewall rule limited access to the Win10 address and was deleted afterwards.

Win10's DNS server was then pointed at Kali. INetSim was started to answer on the common service ports. Its own DNS service would not stay up, so dnsmasq took over that job and replies to every name with 192.168.56.10. From the malware's point of view, every domain on the internet resolves to one obliging server. For runs 2 and 3, a firewall rule also redirected port 587 to INetSim's mail service on port 25, so that mail connections could complete. A test from Win10 confirmed the setup (Figure 1).

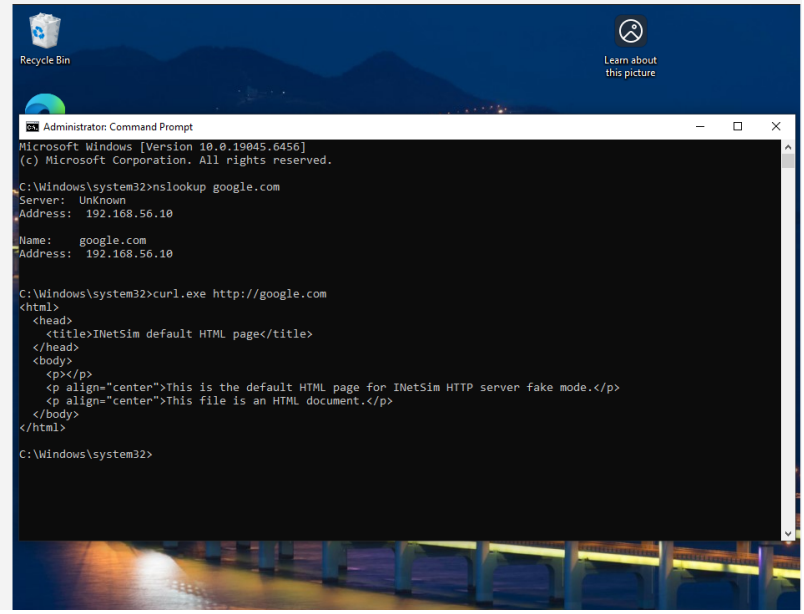


Figure 1. Lab network test from Win10: nslookup resolves google.com to Kali, and curl returns INetSim's default page.

This setup has one consequence for the evidence. Every address in the captures that reads 192.168.56.10 stands in for whatever real server the sample was after. The real destination is known only from the name the sample asked DNS for.

3. The sample

A real executable was chosen on purpose. Many of the AgentTesla entries on MalwareBazaar are small script files that only download the real payload, and in an offline lab they would have shown very little. The one selected is a Windows executable with the AgentTesla signature (Figure 2).

Property	Value
Name on MalwareBazaar	Shipment_Invoice_and_Packing_List.com
SHA256	e76c5c79066d31f5e84b172d61ba575926a18505efa47a9d9a0c7c1c0ced4768
SHA1	6eddd24221c19404b63350c2aee80bf919577d69
MD5	0a7527626bfbc9702ad9fe8428889361
File size	784,384 bytes (about 766 KB)
File type	Windows executable (PE) containing .NET code
First seen	2026-10-01 04:03:48 UTC
Signature	AgentTesla, with 18 vendor detections at the time

Property	Value
Reporter	threatcat_ch

Intelligence 18 | IOCs | YARA 5 | File information | Comments | Actions

SHA256 hash:	e76c5c79066d31f5e84b172d61ba575926a18505efa47a9d9a0c7c1c0ced4768
SHA3-384 hash:	258ae54b8c8e6660ddd7f5af6758bdf8abf743f3b9d99dc478403eb6ba967cafeb84d4b8bd37c02d984fbf26b9144ece
SHA1 hash:	6eddd24221c19404b63350c2aee80bf919577d69
MD5 hash:	0a7527626bfbc9702ad9fe8428889361
humanhash:	alabama-four-papa-east
File name:	Shipment_Invoice_and_Packing_List.com
Download:	download sample
Signature	AgentTesla Alert
File size:	784'384 bytes
First seen:	2026-10-01 04:03:48 UTC
Last seen:	*Never*
File type:	exe
MIME type:	application/x-dosexec
imphash	f34d5f2d4577ed6d9ceec516c1f5a744 (49'247 x AgentTesla, 20'517 x Formbook, 12'379 x SnakeKeylogger)
ssdeep	12288:ncObdIm/X5xiMbSNk5Wpvnbzf4hu6yoRH+EkEMzEqBlvZO1XTHwZv/vztBYto2DN:cOOSR1n/4hDLHoEfcONcxb06

Figure 2. MalwareBazaar entry for the sample: file name, hashes, size, first-seen date and signature.

The sample was downloaded inside the Kali VM, never on the host, as a zip archive protected by the password “infected”. It was moved to Win10 over the host-only link and extracted there. Before it was run, its SHA256 was calculated on Win10 and compared with the value on MalwareBazaar. They match exactly (Figure 3), so the file analysed is the one that was documented.

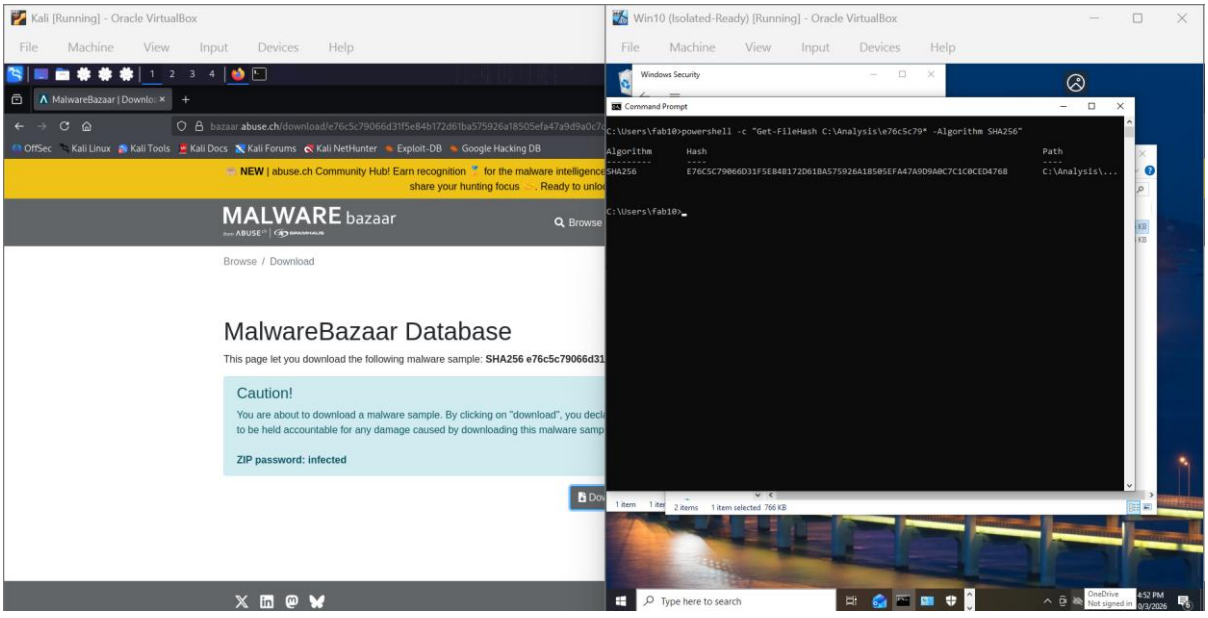


Figure 3. The SHA256 calculated on Win10 (right) matches the MalwareBazaar download page opened on Kali (left).

Two details stand out already. The name is a shipping-paperwork lure that ends in .com, an old executable extension that people rarely associate with programs. And Process Monitor later reported the file’s company as “SemaphoreLine Systems”, a vendor that does not exist, which looks like an attempt to appear ordinary.

4. Method

Each run followed the same routine. Win10 was restored to the clean snapshot. On Win10, Regshot took a first snapshot of the registry and the user folders (run 1), Process Monitor was started and cleared, and on Kali INetSim, dnsmasq and tcpdump were started. The sample was then launched with a double-click, left alone, and observed. At the end the logs were saved and the results compared.

The sample was run three times, and the runs did not produce identical sets of evidence. A Process Monitor log was lost after run 1 because it was not saved before the VM was shut down, run 3 has no packet capture, and run 2's capture was started but not analysed for this report. For that reason each finding below says which run it comes from.

Run	Launch (Win10 clock)	Observed for	Evidence kept	Used for
1	6 Oct 2026, about 11:29 AM	About 30 minutes in the process tree; about an hour of network traffic	Process tree screenshots, Regshot comparison, run1.pcap. Procmon log not saved.	Self-relaunch, registry and file changes, DNS lookups, port 587 attempts
2	16 Oct 2026, 10:48:40 AM	About 2 hours 20 minutes	run2.pml and a process tree screenshot	Long-running process tree
3	16 Oct 2026, about 1:29 PM	Not timed	run3.PML and the INetSim service log. No packet capture.	Linking the port 587 connection to the sample; the mail session

A note on time. The Win10 clock shows Lagos time, and Kali's logs are in US Eastern time, so Win10 runs five hours ahead, plus a drift of a few seconds. Packet capture and INetSim times are Kali time. Because the clocks do not match exactly, the key link in run 3 was made by matching the connection's source port in both logs, not by comparing times.

5. Findings

5.1 Process behavior

In run 2, Process Monitor was set up to keep only events from the sample, from the legitimate .NET programs RegAsm.exe, MSBuild.exe and vbc.exe (which malware of this family is commonly reported to inject into), and with its own noise excluded. In the first second after launch, the sample was reading files under C:\Windows\Microsoft.NET\Framework and registry keys under HKCU\Software\Microsoft\.NETFramework. That is the .NET runtime starting up, which fits a program written for .NET.

The process tree shows the clearest behavior. In run 1, the original process (PID 2868) was started by Explorer at 11:29:33 and exited at 11:30:10, about 37 seconds later. One second before it exited, at 11:30:09, a second process (PID 3964) had started from the same path with the same command line and no arguments. The copy was still running about half an hour later, when the tree was captured at 11:58. Both ran as the normal logged-in user, so no administrator rights were needed.

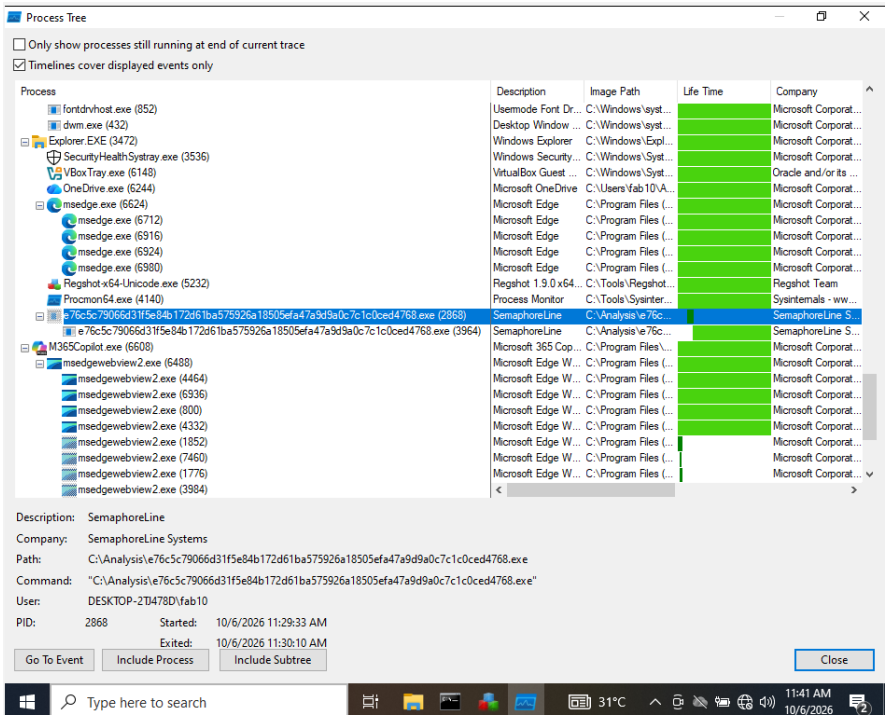


Figure 4. Run 1, process tree. The original (PID 2868) started at 11:29:33 and exited at 11:30:10; its copy (PID 3964) sits underneath. Company and description read SemaphoreLine.

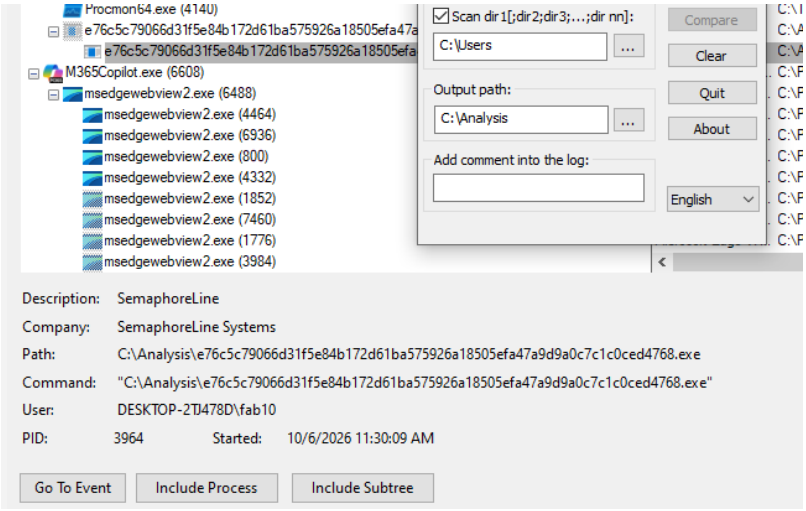


Figure 5. Run 1, details of the copy (PID 3964): started 11:30:09 with the same path, command and user.

Run 2 produced a similar tree after about two hours and twenty minutes: the sample, a copy of it, and a third entry with the same name and no visible parent. None of the three .NET programs being watched for appeared in the tree. Whether the third entry is another relaunch or a leftover from the first handover was not established.

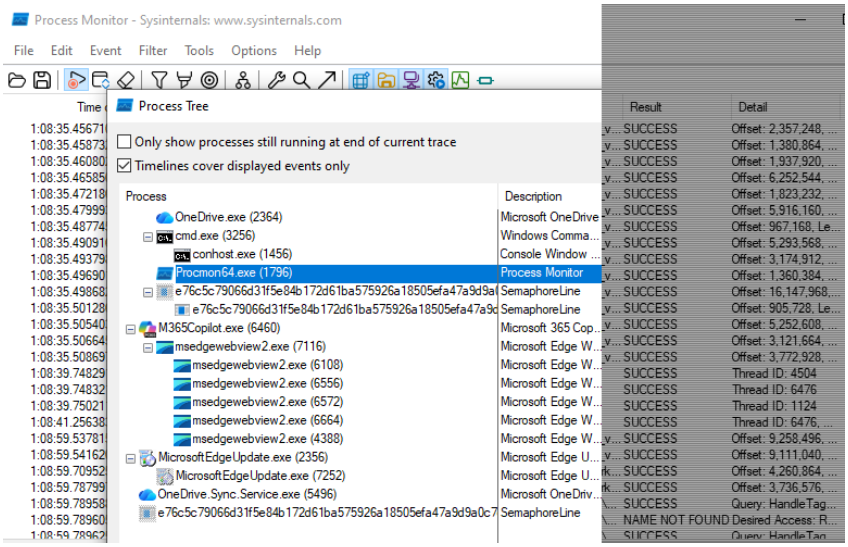


Figure 6. Run 2, process tree about two hours after launch: the original, a copy, and a third entry without a visible parent.

5.2 File system and registry

After run 1, the sample was still sitting in C:\Analysis at 766 KB, unchanged. It had not deleted itself or left a copy in that folder.

Regshot compared the machine about 14 minutes after launch. The comparison reported 6 keys deleted, 60 keys added and 175 values modified, almost all of it Windows housekeeping: provisioning, Group Policy refreshes, error reporting and the disk defragmenter's scheduled statistics. A search of the report for the sample's file name returned a single hit, an entry under the Program Compatibility Assistant store for C:\Analysis\e76c5c79…exe. That entry is Windows recording that the program ran. It is a useful forensic artifact, but Windows wrote it, not the malware. A search for executables under AppData found nothing.

No autostart entry, scheduled task or dropped executable was found in the parts of the report that were checked. Persistence is therefore recorded as **not observed**, not as ruled out, because the autostart and scheduled-task sections were not read line by line.

5.3 Network behavior

Run 1's packet capture shows two kinds of traffic. Most of it is ordinary Windows background activity: Windows Update, Edge, Office and Bing lookups, and web requests to Microsoft servers. One name does not belong. **smtp.yandex.com** was looked up four times, at 06:51:18, 07:11:23, 07:31:15 and 07:51:15 on Kali's clock, which is a gap of about 20 minutes each time. Each lookup was answered a couple of milliseconds later.

```
┌──(obinna㉿kali)-[~]
└─$ tshark -r ~/captures/run1.pcap -Y "dns.qry.name contains yandex" -T fields -e frame.time -e dns.qry.name
2026-10-06T06:51:18.142728000-0400      smtp.yandex.com
2026-10-06T06:51:18.144967000-0400      smtp.yandex.com
2026-10-06T07:11:23.551859000-0400      smtp.yandex.com
2026-10-06T07:11:23.552672000-0400      smtp.yandex.com
2026-10-06T07:31:15.221748000-0400      smtp.yandex.com
2026-10-06T07:31:15.221939000-0400      smtp.yandex.com
2026-10-06T07:51:15.215778000-0400      smtp.yandex.com
2026-10-06T07:51:15.216482000-0400      smtp.yandex.com

tshark: The file "/home/obinna/captures/run1.pcap" appears to have been cut short in the middle of a packet.
```

Figure 7. Run 1 packet capture: lookups for smtp.yandex.com roughly every 20 minutes (Kali clock, UTC-4). Each lookup is followed by its answer.

After each lookup came a burst of four or five packets to port 587 on the stand-in server, and in the later cycles a second burst about a minute afterwards. Port 587 is the standard port for submitting email. In run 1 nothing was listening there, so these connections never completed.

```
┌──(obinna㉿kali)-[~]
└─$ tshark -r ~/captures/run1.pcap -Y "tcp.dstport==25 || tcp.dstport==465 || tcp.dstport==587" -T fields -e frame.time -e ip.dst -e tcp.dstport
2026-10-06T06:51:18.200913000-0400      192.168.56.10   587
2026-10-06T06:51:18.707697000-0400      192.168.56.10   587
2026-10-06T06:51:19.225162000-0400      192.168.56.10   587
2026-10-06T06:51:19.743347000-0400      192.168.56.10   587
2026-10-06T06:51:20.251003000-0400      192.168.56.10   587
2026-10-06T07:11:23.610051000-0400      192.168.56.10   587
2026-10-06T07:11:24.199391000-0400      192.168.56.10   587
2026-10-06T07:11:24.746141000-0400      192.168.56.10   587
2026-10-06T07:11:25.433636000-0400      192.168.56.10   587
2026-10-06T07:11:28.502291000-0400      192.168.56.10   587
2026-10-06T07:12:18.300199000-0400      192.168.56.10   587
2026-10-06T07:12:18.835312000-0400      192.168.56.10   587
2026-10-06T07:12:19.362113000-0400      192.168.56.10   587
2026-10-06T07:12:19.881528000-0400      192.168.56.10   587
2026-10-06T07:12:20.407797000-0400      192.168.56.10   587
2026-10-06T07:31:15.229985000-0400      192.168.56.10   587
2026-10-06T07:31:15.744912000-0400      192.168.56.10   587
2026-10-06T07:31:16.267761000-0400      192.168.56.10   587
2026-10-06T07:31:16.785231000-0400      192.168.56.10   587
2026-10-06T07:31:17.314457000-0400      192.168.56.10   587
2026-10-06T07:32:17.608786000-0400      192.168.56.10   587
2026-10-06T07:32:18.109617000-0400      192.168.56.10   587
2026-10-06T07:32:18.613791000-0400      192.168.56.10   587
2026-10-06T07:32:19.141454000-0400      192.168.56.10   587
2026-10-06T07:32:19.658403000-0400      192.168.56.10   587
2026-10-06T07:51:15.226613000-0400      192.168.56.10   587
2026-10-06T07:51:15.730855000-0400      192.168.56.10   587
2026-10-06T07:51:16.248724000-0400      192.168.56.10   587
2026-10-06T07:51:16.755396000-0400      192.168.56.10   587
2026-10-06T07:51:17.257391000-0400      192.168.56.10   587
2026-10-06T07:52:17.640186000-0400      192.168.56.10   587
2026-10-06T07:52:18.142391000-0400      192.168.56.10   587
2026-10-06T07:52:18.672671000-0400      192.168.56.10   587
2026-10-06T07:52:19.196430000-0400      192.168.56.10   587
```

Figure 8. Run 1 packet capture: bursts of packets to port 587 on the stand-in server (Kali clock, UTC-4).

A regular rhythm like this, from a machine nobody was using, is what is called beaconing. It matches how AgentTesla is generally described, which is that it collects data on the victim's computer and sends it out by email on a schedule.

Run 3 tied the behavior to the sample. With port 587 now redirected to INetSim's mail service, Process Monitor was filtered to show connections to port 587 from every process. Only one event matched: the sample (PID 4748), from source port 54490, at 1:29:37 PM, result SUCCESS.

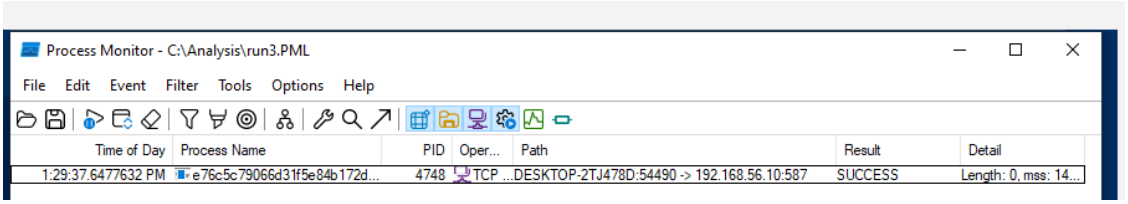


Figure 9. Run 3, Process Monitor filtered on port 587: the only match is the sample (PID 4748).

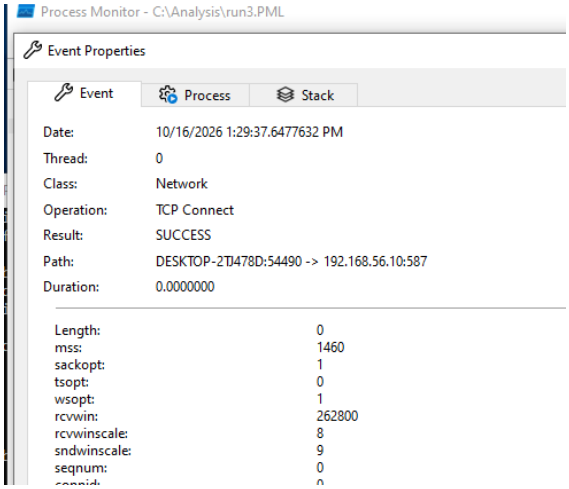


Figure 10. Run 3, event properties of that connection: source port 54490 to 192.168.56.10:587.

INetSim logged the same connection. Searching its log for source port 54490 finds session 119225. The client greets the server with the Win10 machine's own name, is offered the server's capabilities including STARTTLS, asks for STARTTLS, and the connection is upgraded to encryption. About two seconds after it began, the client disconnects. Later sessions in the same log, from the same machine, follow the same pattern.

```
┌──(obinna㉿kali)-[~]
└─$ sudo grep -a "192.168.56.20:54490" /var/log/inetsim/service.log | head -20
[2026-10-16 08:29:53] [5531] [smtp_25_tcp 119225] [192.168.56.20:54490] connect
[2026-10-16 08:29:54] [5531] [smtp_25_tcp 119225] [192.168.56.20:54490] send: 220 mail.inetsim.org INetSim Mail Service ready.
[2026-10-16 08:29:54] [5531] [smtp_25_tcp 119225] [192.168.56.20:54490] recv: EHLO DESKTOP-2TJ478D
[2026-10-16 08:29:54] [5531] [smtp_25_tcp 119225] [192.168.56.20:54490] send: 250-mail.inetsim.org
[2026-10-16 08:29:54] [5531] [smtp_25_tcp 119225] [192.168.56.20:54490] send: 250-SIZE 102400000
[2026-10-16 08:29:54] [5531] [smtp_25_tcp 119225] [192.168.56.20:54490] send: 250-ENHANCEDSTATUSCODES
[2026-10-16 08:29:54] [5531] [smtp_25_tcp 119225] [192.168.56.20:54490] send: 250-VRFY
[2026-10-16 08:29:54] [5531] [smtp_25_tcp 119225] [192.168.56.20:54490] send: 250-DSN
[2026-10-16 08:29:54] [5531] [smtp_25_tcp 119225] [192.168.56.20:54490] send: 250-STARTTLS
[2026-10-16 08:29:54] [5531] [smtp_25_tcp 119225] [192.168.56.20:54490] send: 250-ETRN
[2026-10-16 08:29:54] [5531] [smtp_25_tcp 119225] [192.168.56.20:54490] send: 250-8BITMIME
[2026-10-16 08:29:54] [5531] [smtp_25_tcp 119225] [192.168.56.20:54490] send: 250-EXPN
[2026-10-16 08:29:54] [5531] [smtp_25_tcp 119225] [192.168.56.20:54490] send: 250-AUTH PLAIN LOGIN ANONYMOUS CRAM-MD5 CRAM-SHA1
[2026-10-16 08:29:54] [5531] [smtp_25_tcp 119225] [192.168.56.20:54490] send: 250 HELP
[2026-10-16 08:29:54] [5531] [smtp_25_tcp 119225] [192.168.56.20:54490] recv: STARTTLS
[2026-10-16 08:29:54] [5531] [smtp_25_tcp 119225] [192.168.56.20:54490] send: 220 Ready to start TLS
[2026-10-16 08:29:54] [5531] [smtp_25_tcp 119225] [192.168.56.20:54490] info: Connection successfully upgraded to SSL
[2026-10-16 08:29:55] [5531] [smtp_25_tcp 119225] [192.168.56.20:54490] disconnect
[2026-10-16 08:29:55] [5531] [smtp_25_tcp 119225] [192.168.56.20:54490] stat: 0

┌──(obinna㉿kali)-[~]
└─$
```

Figure 11. INetSim's log for source port 54490: EHLO, STARTTLS, upgrade to SSL, then disconnect after about two seconds (Kali clock).

Everything after the upgrade is encrypted, so INetSim could not record what was said, and its mailbox files stayed empty. What can be stated is that the sample opened an SMTP connection, asked for encryption, completed the upgrade, and ended the session. What cannot be stated is that any data was sent.

The sample's second connection in run 3, at 1:29:39 PM from source port 54492, went to port 80. It is not a request made by the malware. INetSim's log shows a request for a certificate trust list to ctldl.windowsupdate.com from the Microsoft CryptoAPI. Windows makes that request when a program checks a security certificate, and Process Monitor blames the sample because the check runs inside the calling program. It happened at the same moment as the encrypted handshake, which means it was most likely triggered by the sample checking the simulated server's certificate. The mail session ended in the same second. One reading is that the sample did not trust the self-signed certificate and hung up. That is an inference from the timing and is not proven by the logs.

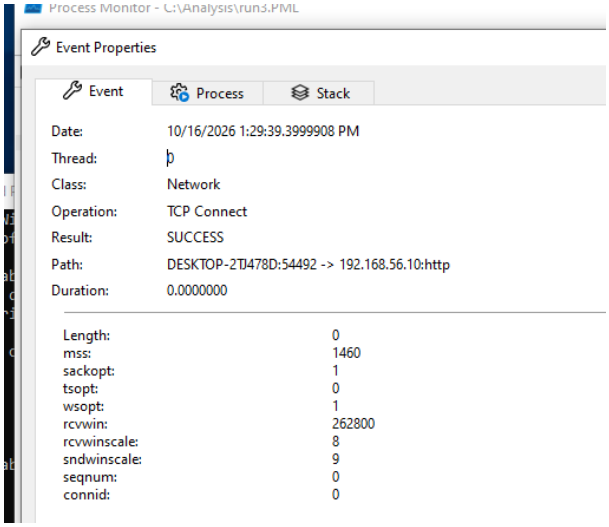


Figure 12. Run 3, the sample's second connection (source port 54492) went to port 80.

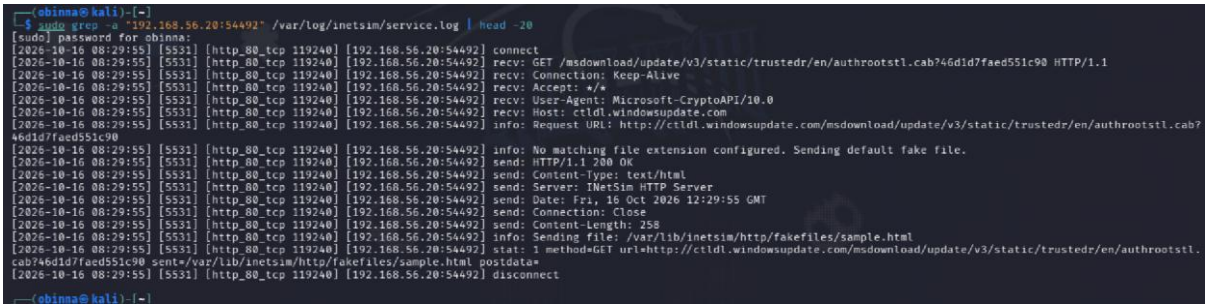


Figure 13. INetSim's log for that connection: a Windows CryptoAPI request for a certificate trust list, not a web request made by the malware.

Run 1's capture agrees. Every web request in it belongs to Windows or Microsoft services. There was no lookup site, no download server and no other call to an outside address that could be tied to the sample, so the mail server is the only destination attributed to it.

5.4 What was not seen

No stolen data was observed, and nothing in these runs showed keylogging, screenshots or credential theft directly. The lab shows how the sample behaves around the network, not what it collects.

6. Classification

The sample is classified as **AgentTesla, an information stealer** (spyware), delivered as a disguised .NET executable. The family label comes from MalwareBazaar, where it carries the AgentTesla signature and 18 vendor detections. The behavior seen in the lab is consistent with that label. AgentTesla is sometimes also called a remote access trojan, but nothing seen in these runs involved remote control, so "information stealer" is the better description of what was observed.

Observation	Why it matters
Invoice-style name ending in .com, fake vendor details ("SemaphoreLine Systems")	Masquerading as ordinary paperwork and software. Typical of a malicious email attachment.

Observation	Why it matters
Managed .NET code, runtime loaded at launch	AgentTesla is written in .NET.
Starts a copy of itself, then the original exits	Hands over to a background process, a common way to hide the launch.
Lookups of smtp.yandex.com about every 20 minutes, then connections to port 587	Scheduled contact with a mail server, matching how the family sends out what it collects.
Only the sample connects to port 587, and it asks for STARTTLS	Ties the mail traffic to the malware and shows it expects an encrypted mail session.
No sign of file encryption, a ransom note or scanning for other machines	Does not look like ransomware or a worm. Observation was limited, so this is not a guarantee.

Confidence in the family is high, because the label, the detection count and the behavior all agree. Confidence in the exact role, stealer rather than something broader, is moderate, because the stolen data itself was never seen.

7. Indicators of compromise

The network indicators below were observed in a lab where every domain was redirected to a simulated server, so the real destination was never contacted. “MB” means the value comes from MalwareBazaar.

Type	Indicator	Notes	Run
SHA256	`e76c5c79066d31f5e84b172d61ba575926a18505efa47a9d9a0c7c1c0ced4768`	Verified on Win10	All
SHA1	`6eddd24221c19404b63350c2aee80bf919577d69`	MB	n/a
MD5	`0a7527626bfbc9702ad9fe8428889361`	MB	n/a
File name	`Shipment_Invoice_and_Packing_List.com`	Lure name. 784,384 bytes.	MB
File metadata	`Company “SemaphoreLine Systems”, description “SemaphoreLine”`	Not a real vendor. Shown by Process Monitor.	1, 3
Domain	`smtp.yandex.com`	Looked up about every 20 minutes. Legitimate service used as the destination.	1
Network	`TCP port 587 with STARTTLS`	Only the sample connected to port 587 in run 3.	1, 3

Type	Indicator	Notes	Run
Behavior	`Launches a copy of itself from the same path; the original exits within about a minute`	Self-relaunch.	1, 2
Behavior	`.NET runtime loaded at launch`	Managed code.	2
imphash	`f34d5f2d4577ed6d9ceec516c1f5a744`	Weak. Shared with Formbook and SnakeKeylogger samples.	MB

Left out on purpose: ctldl.windowsupdate.com and the other Microsoft domains (normal Windows traffic), the Compatibility Assistant registry entry (Windows recording that the file ran), the name DESKTOP-2TJ478D (the lab machine's own name), and the lab addresses 192.168.56.10 and 192.168.56.20.

8. Threat actor hypothesis

What the evidence supports. AgentTesla is sold and cracked widely, and many unrelated criminals use it, so the family name says little about who ran this particular copy. What this copy does show is an invoice and packing-list lure name, a made-up vendor identity, and an account on a free webmail service as the route out. That combination is typical of opportunistic, financially motivated campaigns. The lure suggests the intended victims handle shipping and trade paperwork, such as small businesses and logistics staff.

Hypothesis. A low-sophistication, financially motivated operator is running a mass phishing campaign with commodity tools, collecting saved logins and other data from infected machines for fraud or resale.

Confidence. Moderate for the motive and the level of sophistication, low for any specific attribution. A particular group cannot be named from one sample and a few hours of observation.

What the lab could not answer. Who owns the mailbox, how the file was delivered, and which sector was targeted. Confidence would go up with the sender and recipient addresses and the mailbox login stored in the sample, the delivering email and its headers, and other samples that use the same mailbox. The first of these could be recovered by decoding the sample's configuration or by getting the encrypted mail session decrypted.

MITRE ATT&CK (suggested mapping)	Basis
T1036 Masquerading	Invoice-style .com name and invented vendor details. Observed.
T1071.003 Application Layer Protocol: Mail Protocols	Repeated SMTP connections to the mail submission port. Observed.
T1048 Exfiltration Over Alternative Protocol	Consistent with the family's behavior. Not observed, since no data content was seen.

9. Detection and defence recommendations

Measure	What it addresses
Block outbound SMTP (ports 25, 465 and 587) from ordinary workstations, allowing only mail servers	Stops this exfiltration route and makes any attempt a clear alert.
Alert on DNS lookups for webmail SMTP hostnames from machines that do not send mail, especially at regular intervals	Catches the beaconing seen in run 1.
Filter email for invoice-style attachments, double or unusual extensions such as .com, and password-protected archives	Targets the delivery lure.
Block executables run from user-writable folders and downloads, using application control	Stops the file from running at all.
Detect a .NET program that starts a copy of itself and then exits, or a user process making SMTP connections	Behavior rules for endpoint tools, independent of the file hash.
Treat any infection as a credential incident: isolate the machine, change saved passwords and enable multi-factor authentication	Stealers collect saved logins, so cleaning the machine is not enough.

10. Problems met along the way

A few things went wrong in the lab, and the fixes are worth recording because they affect how far the results can be trusted.

- **INetSim's DNS service.** The log said it had started, but nothing was listening on port 53. Checking the listening ports showed the gap, and dnsmasq replaced it.
- **Firewall on Kali.** The file transfer and the DNS replies were both blocked until a rule for the Win10 address was added. The transfer rule was removed afterwards.
- **Reading the Regshot report.** The first searches came back empty because the report is stored as UTF-16 and the search tool could not read it. Searching for a value already known to be in the file showed the problem, and the right encoding fixed it. The lesson is to test a search on a known value before trusting an empty result.
- **A lost log.** Run 1's Process Monitor log was lost when the VM was shut down before it was saved. Later runs saved the log straight away, and the filters were set before launch to keep it small.
- **Mismatched clocks.** The two machines' clocks differed by five hours and a few seconds, so events were matched by source port, not by time.

11. Limitations

- One sample, three runs, and observation windows far shorter than a real infection.
- The mail session was encrypted, so no stolen data was seen. The idea that the sample rejected the simulated server's certificate comes from timing and is not proven.

- The evidence is uneven between runs, as set out in Section 4, which is why each finding names its run.
- Persistence was not ruled out. The autostart, scheduled-task and added-files sections of the Regshot report were not fully reviewed.
- Defender was off and the machine was a virtual one. Many samples check for sandboxes and may stay quiet or behave differently, so a different environment could show more.
- The analysis is based on behavior only. The executable was not unpacked or decompiled.
- Destinations were simulated, so the real mail server's response was never seen.

12. Conclusion

The sample behaves as an AgentTesla information stealer. It arrives as an invoice-style executable with invented vendor details, launches a copy of itself and drops the original, and then contacts a mail server on the submission port at regular intervals. In the best-documented run, Process Monitor and the simulated mail server together show that the sample, and no other process, opened an SMTP session, asked for encryption, and ended the session. What it was trying to send could not be read.

The strongest indicators for defenders are the file hashes, the repeated lookups of smtp.yandex.com from a machine that does not send mail, and the self-relaunch pattern. The actor is most plausibly a financially motivated criminal using commodity tooling, and the data does not support naming a group. The lab's gaps, mainly the encrypted mail content and the unevenness between runs, are stated openly, and a final run with captures and logs recording at the same time would be the natural next step.

Appendix A. Evidence inventory

Item	Run	What it shows
MalwareBazaar entry and hash comparison (Figures 2 and 3)	Setup	Identity of the sample
Lab network test (Figure 1)	Setup	DNS and the simulated services work
Process tree screenshots (Figures 4 and 5)	1	Self-relaunch, fake metadata
Process tree screenshot (Figure 6), run2.pml	2	Long-running process tree
regshot_run1.txt	1	Registry and user-folder changes
run1.pcap (Figures 7 and 8)	1	smtp.yandex.com lookups, port 587 attempts
run3.PML (Figures 9, 10 and 12)	3	Connections made by the sample
INetSim service log (Figures 11 and 13)	3	SMTP session, certificate trust list request

Appendix B. Key commands

Hash of the sample on Win10:

```
powershell -c "Get-FileHash C:\Analysis\e76c5c79* -Algorithm SHA256"
```

Fake DNS on Kali, answering every name with Kali's address:

```
sudo dnsmasq --no-daemon --listen-address=192.168.56.10 --bind-interfaces --no-resolv --
address=/#/192.168.56.10
```

Redirect port 587 to INetSim's mail service:

```
sudo iptables -t nat -A PREROUTING -i eth0 -p tcp --dport 587 -j REDIRECT --to-ports 25
```

Domains looked up by Win10 in the capture, and the smtp.yandex.com lookups with times:

```
tshark -r ~/captures/run1.pcap -Y "dns.flags.response==0" -T fields -e dns.qry.name | sort |
uniq -c | sort -rn | head -30
tshark -r ~/captures/run1.pcap -Y "dns.qry.name contains yandex" -T fields -e frame.time -e
dns.qry.name
```

Packets to the mail ports:

```
tshark -r ~/captures/run1.pcap -Y "tcp.dstport==25 || tcp.dstport==465 || tcp.dstport==587" -T
fields -e frame.time -e ip.dst -e tcp.dstport
```

INetSim's log for the run 3 connection, found by source port:

```
sudo grep -a "192.168.56.20:54490" /var/log/inetsim/service.log
```

Searching the UTF-16 Regshot report:

```
Select-String -Path C:\Analysis\regshot_run1.txt -Encoding Unicode -SimpleMatch -Pattern
'e76c5c79'
```